

Veille Technologique & TP

Intégration d'Arduino, Node.js et MongoDB

Présenté par :

- Makhtar WADE
 - Mouhamed DIOP
 - Bintou DIAME
-

Juin 2026

Qu'est-ce que MongoDB ?

C'est une base de données moderne, conçue pour stocker des volumes importants de données variées. Contrairement aux bases classiques :

- Elle n'utilise pas de tables rigides ni de colonnes.
- Elle stocke les informations sous forme de **documents** (très similaires à des objets JSON).
- Elle offre une grande rapidité d'écriture, ce qui est parfait pour enregistrer des mesures de capteurs arrivant en continu.

Quelle est la différence entre une base de données relationnelle et une base de données NoSQL ?

La différence se joue sur la flexibilité et la structure :

- **Base relationnelle (SQL)** : Structure rigide. Les données sont organisées dans des tables fixes liées par des clés. Si on veut modifier la structure, il faut modifier toute la table.
- **Base NoSQL (ex: MongoDB)** : Structure libre. Il n'y a pas de schéma imposé. Chaque enregistrement peut contenir des champs différents de son voisin, ce qui permet d'adapter facilement la base aux évolutions du projet.

Pourquoi MongoDB est-elle considérée comme une base de données orientée documents ?

Parce que l'élément fondamental de stockage est le **document**.

- Au lieu de découper une information sur plusieurs tables (ex: une table capteur, une table historique), on stocke tout ce qui concerne un objet au même endroit, dans un seul document.
- Cela évite de faire des requêtes complexes avec des jointures (JOIN), ce qui accélère la lecture et simplifie le code.

Qu'est-ce qu'un document dans MongoDB ?

C'est l'équivalent d'une ligne dans une table SQL, mais sous forme d'un ensemble de clés et de valeurs (comme un objet en JavaScript).

- **Flexibilité** : Il peut contenir des textes, des nombres, des tableaux, ou même d'autres objets imbriqués.
- Exemple simple :

```
{  
  "sensor_name": "DHT11",  
  "values": { "temp": 24.2, "hum": 60 }  
}
```

Qu'est-ce qu'une collection ?

Une collection est un regroupement de documents. C'est l'équivalent d'une **table** en SQL.

- Une base de données peut contenir plusieurs collections (ex: `users`, `sensor_data`, `alerts`).
- La particularité est qu'elle n'impose aucune contrainte de forme : vous pouvez mettre deux documents de structures différentes dans la même collection sans problème.

Quel format de données MongoDB utilise-t-elle ?

MongoDB utilise deux formats complémentaires :

- **JSON** : Pour l'utilisateur. C'est le format texte sous lequel on écrit nos requêtes et on lit nos résultats.
- **BSON (Binary JSON)** : Pour le stockage interne. C'est une version binaire compressée du JSON, optimisée pour être lue très rapidement par la machine.

Quelle est la différence entre JSON et BSON ?

Bien qu'ils se ressemblent, ils ont des rôles différents :

- **JSON** est un format texte lisible par l'homme, mais limité en types de données (pas de distinction entre entier et décimal, pas de type Date natif).
- **BSON** est binaire (invisible pour l'humain), plus compact, et ajoute des types précis indispensables (dates réelles, identifiants uniques `ObjectId`, données binaires brutes).

Pourquoi MongoDB peut-elle être adaptée à une solution IoT ?

- **Schéma évolutif** : Si vous rajoutez un capteur de luminosité à votre projet, vous pouvez envoyer cette donnée directement à la base sans configurer de nouvelles tables.
- **Rapidité d'écriture** : Les objets connectés génèrent beaucoup de données en continu. MongoDB encaisse très bien cette charge.
- **Compatibilité avec JavaScript** : Node.js (côté serveur) et MongoDB partagent la même logique d'objets, ce qui rend le développement fluide.

Quels types de données provenant d'un Arduino peut-on enregistrer dans MongoDB ?

- **Mesures physiques** : Température, humidité, niveau de luminosité, distance d'un obstacle.
- **États de composants** : Alarme activée (vrai/faux), présence détectée (0 ou 1).
- **Métadonnées du système** : Identifiant de la carte (adresse MAC), tension d'alimentation, niveau du signal Wi-Fi.
- **Horodatage** : L'heure exacte de la capture.

Comment une application Node.js peut-elle communiquer avec MongoDB ?

La communication s'organise avec des bibliothèques JavaScript :

1. Node.js utilise un **pilote de connexion** (driver) pour se connecter à la base via une chaîne de connexion (URI).
2. Pour simplifier l'écriture des requêtes et structurer le code, on utilise un outil appelé **Mongoose**.
3. On utilise des commandes simples comme `.find()` pour lire et `.save()` pour écrire.

Quel est le rôle de la bibliothèque Mongoose ?

Mongoose sert d'intermédiaire (un ODM) entre Node.js et MongoDB pour :

- **Valider les données** : S'assurer que le capteur envoie bien un nombre et pas du texte avant d'enregistrer.
- **Structurer le code** : Définir à l'avance des modèles pour chaque type d'information.
- **Faciliter les requêtes** : Éviter d'écrire des requêtes trop techniques en proposant des fonctions JavaScript simples.

Qu'est-ce qu'un schéma Mongoose ?

C'est le plan de construction de vos documents. Il définit exactement la structure que doivent respecter les données de votre collection.

- Il définit les **champs**, leurs **types** (String, Number, Date...) et les contraintes (ex: champ obligatoire).
- Exemple :

```
const sensorSchema = new mongoose.Schema({  
  sensor_id: { type: String, required: true },  
  value: Number  
});
```

Comment enregistrer les données d'un capteur dans MongoDB ?

On utilise le modèle Mongoose pour créer un nouvel enregistrement et l'envoyer en base :

```
// 1. On prépare la donnée
const nouvelleMesure = new Sensor({
  sensor_id: "DHT11",
  value: 23.5
});

// 2. On l'enregistre en base
await nouvelleMesure.save();
```

Comment rechercher les données envoyées par un objet connecté ?

On utilise la fonction `.find()` avec des critères de sélection :

```
// Trouver toutes les mesures du capteur DHT11
const mesures = await Sensor.find({ sensor_id: "DHT11" });

// Trouver les mesures supérieures à 25°C
const alertes = await Sensor.find({ value: { $gt: 25 } });
```

Comment modifier ou supprimer une donnée dans MongoDB ?

On utilise des méthodes ciblées de Mongoose :

```
// Pour modifier une valeur (ex: changer le nom d'un capteur)
await Sensor.updateOne({ sensor_id: "DHT11" }, { sensor_id: "DHT11_Salle1" });

// Pour supprimer des mesures obsolètes
await Sensor.deleteMany({ value: 0 });
```


Comment enregistrer automatiquement la date et l'heure d'une mesure ?

Il y a deux méthodes simples dans le schéma Mongoose :

1. Valeur par défaut :

`timestamp: { type: Date, default: Date.now }` (enregistre l'heure de création).

2. Option automatique : Ajouter `{ timestamps: true }` à la fin du schéma pour créer automatiquement les champs `createdAt` (date de création) et `updatedAt` (date de modification).

Quelle différence existe-t-il entre MongoDB local et MongoDB Atlas ?

- **MongoDB Local** : Installé directement sur votre machine. Utile pour travailler hors-ligne et faire des tests rapides sans connexion internet.
- **MongoDB Atlas** : Version hébergée dans le cloud par MongoDB. Accessible de n'importe où, gérée automatiquement, hautement sécurisée, mais nécessite une connexion internet constante.

Quelles sont les limites de MongoDB dans une application IoT ?

- **Taille sur le disque** : Comme chaque document contient le nom de ses clés (ex: "temperature"), cela prend plus de place qu'une base de données optimisée "séries temporelles".
- **Pas pour les microcontrôleurs** : MongoDB est trop lourd pour tourner directement sur une carte Arduino. Il faut obligatoirement un serveur intermédiaire (Node.js).

Comment sécuriser l'accès à une base MongoDB ?

- **Activer l'authentification** : Protéger la base avec un couple utilisateur/mot de passe robuste.
- **Autoriser uniquement certaines adresses IP** (Whitelist) : Bloquer toutes les connexions qui ne viennent pas de votre serveur Node.js.
- **Chiffrer les flux** : Utiliser des connexions sécurisées SSL/TLS.

Pourquoi faut-il protéger le nom d'utilisateur, le mot de passe et l'URI de connexion ?

L'URI de connexion contient toutes les informations d'accès en clair.

- Si un intrus récupère cette ligne, il peut se connecter directement à votre base, **voler vos données**, modifier vos configurations ou **supprimer toute votre base** de données à distance.
- Il ne faut jamais mettre ces informations en clair dans le code envoyé sur GitHub.

Comment utiliser un fichier .env pour protéger les informations de connexion ?

On sépare la configuration du code :

1. On écrit nos secrets dans un fichier local nommé `.env` :

```
MONGO_URI=mongodb://mon_user:mon_mdp@localhost:27017/mabase
```

2. On charge ce fichier dans notre code Node.js avec `dotenv` :

```
require('dotenv').config();  
const uri = process.env.MONGO_URI;
```

3. On ajoute le fichier `.env` dans `.gitignore` pour ne pas le publier.

Comment organiser les collections d'une solution IoT ?

L'idéal est de séparer la configuration des mesures physiques :

- **Collection** `devices` : Un document par carte Arduino (son ID, sa pièce, son état en ligne/hors-ligne).
 - **Collection** `measurements` : Un enregistrement par mesure contenant uniquement la valeur, la date et l'ID de la carte correspondante.
- Cela évite de répéter inutilement les infos de la carte à chaque mesure.

Comment utiliser Socket.IO avec MongoDB dans une application IoT en temps réel ?

Socket.IO sert de pont instantané :

1. L'Arduino envoie une mesure à Node.js.
2. Node.js fait deux actions en parallèle :
 - Il **sauvegarde** la mesure dans MongoDB (pour l'historique).
 - Il **émet** immédiatement la mesure via Socket.IO à tous les navigateurs web connectés.
3. La page web se met à jour en temps réel sans rechargement.

Quelle architecture peut relier Arduino, Node.js, MongoDB et une interface web ?

Le flux d'information suit ce parcours :

1. **Arduino** capte la valeur → Envoi par liaison série (USB).
2. **Node.js** écoute le port série → Reçoit la valeur.
3. **MongoDB** enregistre la valeur pour l'historique.
4. **Socket.IO** envoie la valeur en direct vers l'**interface Web** (graphique dynamique).

Activité pratique — Objectifs & Étapes

Voici notre plan d'action pour la démonstration :

1. Lancement de **MongoDB local** et connexion avec **MongoDB Compass**.
2. Création de la base `iot_database` et de la collection `sensor_data`.
3. Programmation du serveur **Node.js** avec Mongoose.
4. Câblage du capteur sur l'**Arduino**.
5. Enregistrement automatique en base **toutes les 3 minutes** avec horodatage.

Conclusion — Points clés

- **NoSQL / MongoDB** : Apporte la flexibilité indispensable pour gérer des données IoT variables.
- **Sécurité** : Utiliser impérativement des fichiers `.env` pour masquer les identifiants de la base de données.
- **Architecture** : Associer Node.js, MongoDB et Socket.IO permet d'allier stockage à long terme et affichage fluide en temps réel.

Merci pour votre attention. Place maintenant à la démonstration pratique !